

FRAMEWORK COMPLETO DE PREPROCESAMIENTO DE DATOS

Guía Metodológica para Análisis de Machine Learning

FASE 0: COMPRENSIÓN DEL PROBLEMA

Análisis Inicial:

- ✓ **Definición del objetivo** (clasificación, regresión, clustering)
- ✓ **Comprensión del dominio** (contexto del negocio/investigación)
- ✓ **Identificación de restricciones** (tiempo, recursos, interpretabilidad)
- ✓ **Definición de métricas de éxito** (accuracy, F1, ROC-AUC, RMSE, etc.)

Exploración Preliminar:

- ✓ **Carga y visualización inicial** (primeras filas, info, describe)
- ✓ **Dimensionalidad** (número de filas, columnas, memoria)
- ✓ **Tipos de datos** (numéricos, categóricos, temporales, texto)
- ✓ **Distribución de la variable objetivo** (balanceo de clases)

Tarea	Snippet
Carga y visualización inicial	<pre>df = pd.read_csv('data.csv') df.head() df.tail() df.info() df.describe()</pre>
Dimensionalidad	<pre>df.shape df.memory_usage(deep=True).sum()</pre>
Tipos de datos	<pre>df.dtypes df.astype() df.select_dtypes(include=['object', 'datetime64[ns]']).columns</pre>
Balanceo de clases	<pre>df['target'].value_counts() df['target'].value_counts(normalize=True) df['target'].plot.hist() (para regresión)</pre>

FASE 1: ANÁLISIS EXPLORATORIO DE DATOS (EDA)

1.1 Calidad de Datos:

- ✓ **Valores duplicados** (identificación y tratamiento)
- ✓ **Valores faltantes** (cantidad, distribución, patrones)
- ✓ **Valores atípicos** (detección con IQR, Z-score, isolation forest)
- ✓ **Inconsistencias** (formatos, rangos imposibles, conflictos lógicos)

1.2 Análisis Univariado:

- ✓ **Variables numéricas:**
 - Distribuciones (histogramas, boxplots, violin plots)
 - Estadísticas descriptivas (media, mediana, std, min, max)
 - Asimetría y curtosis
 - Detección de outliers
- ✓ **Variables categóricas:**
 - Frecuencias y porcentajes
 - Cardinalidad (número de categorías únicas)
 - Categorías raras (< 5% de frecuencia)
 - Balance entre categorías

1.3 Análisis Bivariado/Multivariado:

- ✓ **Correlaciones** (Pearson, Spearman, matriz de correlación)
- ✓ **Relación con variable objetivo:**
 - Numéricas: scatter plots, correlación
 - Categóricas: barplots agrupados, chi-cuadrado
- ✓ **Multicolinealidad** (VIF - Variance Inflation Factor)
- ✓ **Interacciones entre variables**

Tarea	Snippet	
1.1 Calidad de Datos		
Valores duplicados	<pre>df.duplicated().sum() df = df.drop_duplicates()</pre>	
Valores faltantes	<pre>df.isnull().sum() df.isna().sum() sns.heatmap(df.isna(), cbar=False)</pre>	

	<code>df.isna().mean()</code>	
Valores atípicos Outliers (IQR)Fórmula IQR/Z-score (NumPy)	<code>Q1=df['x'].quantile(0.25); Q3=df['x'].quantile(0.75); IQR=Q3-Q1 `outliers=df[(df['x']<(Q1-1.5*IQR))`</code>	
Outliers (Z-score)	<code>from scipy import stats z=np.abs(stats.zscore(df['x'])); outliers=df[z>3]</code>	
Outliers (Isolation Forest)	<code>from sklearn.ensemble import IsolationForest iso=IsolationForest(contamination=0.01).fit_predict(d f[['x']]); outliers=df[iso==-1]</code>	
Histograma	<code>sns.histplot(df['x'], kde=True)</code>	
Boxplot	<code>sns.boxplot(y=df['x'])</code>	
Violinplot	<code>sns.violinplot(y=df['x'])</code>	
Descriptivos	<code>df['x'].describe()</code>	
Asimetría/Curtosis	<code>from scipy.stats import skew, kurtosis skew(df['x']), kurtosis(df['x'])</code>	
1.2 Análisis Univariado		
Cardinalidad	<code>df['cat'].nunique() df['cat'].value_counts()</code>	
Categorías raras	<code>vc=df['cat'].value_counts(normalize=True); rare=vc[vc<0.05].index</code>	
Variables numéricas	<code>df['col'].hist(), df['col'].plot.box(), df['col'].skew(), df['col'].kurt()</code>	Pandas, Seaborn
Variables categóricas	<code>df['col'].value_counts(), df['col'].nunique()</code>	Pandas
1.3 Análisis Bivariado/Multivari ado		
Correlaciones	<code>df.corr(method='pearson'), sns.heatmap()</code>	Pandas, Seaborn
Relación con target	<code>sns.scatterplot(), sns.boxplot(), pd.crosstab(), chi2_contingency</code>	Seaborn, SciPy
Multicolinealidad	Función para calcular VIF (usando OLS)	Statsmod els

□ FASE 2: LIMPIEZA DE DATOS

2.1 Limpieza Básica:

- ✓ **Eliminación de espacios** en strings (strip, whitespace)
- ✓ **Normalización de texto:**
 - Conversión a minúsculas/mayúsculas consistente
 - Eliminación de caracteres especiales
 - Corrección de typos comunes
- ✓ **Parseo de formatos** (fechas, monedas, porcentajes)
- ✓ **Conversión de tipos** (object → numeric, datetime)

2.2 Tratamiento de Duplicados:

- ✓ **Identificación** de registros duplicados
- ✓ **Análisis de causa** (error de carga, registros legítimos)
- ✓ **Decisión:** eliminar o mantener
- ✓ **Documentación** de la decisión

2.3 Tratamiento de Inconsistencias:

- ✓ **Rangos imposibles** (edades negativas, fechas futuras)
- ✓ **Conflictos lógicos** (fecha_fin < fecha_inicio)
- ✓ **Categorías mal escritas** (unificación de variantes)
- ✓ **Valores por defecto** problemáticos (999, -1, "unknown")

Tarea	Snippet
2.1 Limpieza Básica	
Eliminación de espacios Strip strings	<code>df['col'] = df['col'].str.strip()</code>
Normalización del texto/Lowercase	<code>df['col'] = df['col'].str.lower()</code>
Remover especiales	<code>df['col'] = df['col'].str.replace('[^A-Za-z0-9]+', '', regex=True)</code>
Parsear fechas	<code>df['fecha'] = pd.to_datetime(df['fecha'], format='%d/%m/%Y', errors='coerce')</code>
Parsear porcentajes	<code>df['pct'] = df['pct'].str.rstrip('%').astype(float)/100</code>
Convertir tipos	<code>df['num'] = pd.to_numeric(df['num'], errors='coerce')</code>
Duplicados	<code>df.drop_duplicates(inplace=True)</code>
Rangos imposibles	<code>df = df[(df['edad']>=0)&(df['edad']<=120)]</code>

Conflictos lógicos	<code>df = df[df['fecha_fin']>=df['fecha_inicio']]</code>
Unificar categorías	<code>df['cat'] = df['cat'].replace({'M':'Mujer', 'H':'Hombre'})</code>

FASE 3: MANEJO DE VALORES FALTANTES

3.1 Análisis de Patrones:

- ✓ **Cantidad y porcentaje** por variable
- ✓ **Visualización** (heatmap de missingness)
- ✓ **Clasificación del mecanismo:**
 - **MCAR** (Missing Completely At Random)
 - **MAR** (Missing At Random)
 - **MNAR** (Missing Not At Random)

3.2 Estrategias de Imputación:

Para Variables Numéricas:

- ✓ **Media/Mediana:** Para MCAR con distribución normal/asimétrica
- ✓ **Moda:** Para variables discretas
- ✓ **Imputación por grupo:** Usando otras variables (ej: edad por género)
- ✓ **Interpolación:** Para series temporales
- ✓ **Modelos predictivos:** KNN, MICE, IterativeImputer
- ✓ **Valor constante:** Cuando el faltante tiene significado (-999, "sin_dato")

Para Variables Categóricas:

- ✓ **Moda:** Categoría más frecuente
- ✓ **Nueva categoría:** "Missing", "Unknown", "No_Aplica"
- ✓ **Imputación por grupo:** Basada en otras categorías
- ✓ **Modelo predictivo:** Clasificador para predecir categoría

3.3 Creación de Indicadores:

- ✓ **Variables dummy** indicando si había valor faltante
- ✓ **Útil cuando la ausencia tiene significado** predictivo

Tarea	Snippet
Matriz missing	<code>msno.matrix(df)</code>
Patrón MCAR	<code>from missingno import nullity_correlation</code> <code>nullity_correlation(df, method='pearson')</code>

Imputar media	<code>df['x'].fillna(df['x'].mean(), inplace=True)</code>
Imputar mediana	<code>df['x'].fillna(df['x'].median(), inplace=True)</code>
Imputar por grupo	<code>df['x']=df.groupby('sexo')['x'].transform(lambda x:x.fillna(x.median()))</code>
KNN imputer	<code>from sklearn.impute import KNNImputer imputer=KNNImputer(n_neighbors=5); df_num=imputer.fit_transform(df_num)</code>
MICE	<code>from sklearn.experimental import enable_iterative_imputer from sklearn.impute import IterativeImputer imp=IterativeImputer(random_state=0); df_num=imp.fit_transform(df_num)</code>
Imputar cat moda	<code>df['cat'].fillna(df['cat'].mode()[0], inplace=True)</code>
Nueva categoría	<code>df['cat'].fillna('Missing', inplace=True)</code>
Indicador missing	<code>df['x_missing'] = df['x'].isna().astype(int)</code>

☺ FASE 4: INGENIERÍA DE CARACTERÍSTICAS (FEATURE ENGINEERING)

4.1 Creación de Nuevas Features:

Desde Variables Existentes:

- ✓ **Combinaciones:** ratios, diferencias, productos
- ✓ **Agregaciones:** sumas, promedios, conteos
- ✓ **Binning:** discretización de continuas (edad → grupos etarios)
- ✓ **Indicadores binarios:** (tiene_x → 0/1)

Desde Variables Temporales:

- ✓ **Extracción:** año, mes, día, día_semana, trimestre
- ✓ **Cíclicas:** sin/cos para hora, mes
- ✓ **Diferencias temporales:** días_desde_evento, antigüedad
- ✓ **Indicadores:** es_fin_semana, es_festivo

Desde Variables de Texto:

- ✓ **Longitud:** caracteres, palabras
- ✓ **Patrones:** presencia de números, mayúsculas, símbolos

- ✓ **TF-IDF, Bag of Words** (si aplica)

Desde Variables Geográficas:

- ✓ **Distancias:** a punto de interés
- ✓ **Regiones:** agrupación por zona
- ✓ **Coordenadas:** conversión a formato útil

4.2 Transformaciones de Variables:

Para Normalidad:

- ✓ **Logarítmica:** $\log(x+1)$ para asimetrías positivas
- ✓ **Raíz cuadrada:** $\sqrt{x}$
- ✓ **Box-Cox / Yeo-Johnson:** transformación paramétrica
- ✓ **Inversa:** $1/x$

Para Reducir Impacto de Outliers:

- ✓ **Winsorización:** cap en percentiles
- ✓ **Clipping:** límites mínimo/máximo
- ✓ **Transformaciones robustas**

4.3 Agrupación y Simplificación:

- ✓ **Categorías raras** → "Otros" (< 5% frecuencia)
- ✓ **Niveles jerárquicos** → agrupación a nivel superior
- ✓ **Simplificación conceptual** → menos categorías más significativas

Tarea	Snippet
Ratio	<code>df['ratio'] = df['a'] / (df['b']+1e-6)</code>
Binning	<code>df['edad_grp'] = pd.cut(df['edad'], bins=[0,18,35,60,100], labels=['<18', '18-35', '35-60', '60+'])</code>
Dummy binario	<code>df['tiene_hijos'] = (df['hijos']>0).astype(int)</code>
Extraer año	<code>df['anio'] = df['fecha'].dt.year</code>
Variables cíclicas	<code>df['mes_sin'] = np.sin(2*np.pi*df['mes']/12)</code> <code>df['mes_cos'] = np.cos(2*np.pi*df['mes']/12)</code>
Días desde evento	<code>df['dias_reg'] = (pd.Timestamp('today')-df['fecha']).dt.days</code>
Longitud texto	<code>df['len_name'] = df['name'].str.len()</code>
TF-IDF	<code>from sklearn.feature_extraction.text import TfidfVectorizer</code>

	<pre>tfidf=TfidfVectorizer(max_features=100); X_text=tfidf.fit_transform(df['texto'])</pre>
Haversine distance	<pre>from sklearn.metrics.pairwise import haversine_distances dist=haversine_distances(np.radians(coords))*6371000</pre>
Logaritmo	<pre>df['x_log'] = np.log1p(df['x'])</pre>
Box-Cox	<pre>from scipy.stats import boxcox df['x_bc'],_=boxcox(df['x']+1)</pre>
Winsorizar	<pre>from scipy.stats import mstats df['x_win']=mstats.winsorize(df['x'], limits=[0.05,0.05])</pre>

FASE 5: ENCODING DE VARIABLES CATEGÓRICAS

5.1 Estrategias por Cardinalidad:

Baja Cardinalidad (2-10 categorías):

- ✓ **One-Hot Encoding:** Crea columna binaria por categoría
 - Ventaja: No introduce orden artificial
 - Desventaja: Aumenta dimensionalidad
 - Usar con: Random Forest, SVM, Neural Networks
- ✓ **Dummy Encoding:** One-hot eliminando una categoría de referencia
 - Evita multicolinealidad perfecta
 - Usar con: Regresión Lineal

Media Cardinalidad (10-50 categorías):

- ✓ **Label Encoding:** Asigna números enteros (0, 1, 2...)
 - Ventaja: No aumenta dimensionalidad
 - Desventaja: Introduce orden artificial
 - Usar con: Tree-based models (Random Forest, XGBoost)
- ✓ **Ordinal Encoding:** Para categorías con orden natural
 - Ejemplo: ["Bajo", "Medio", "Alto"] → [0, 1, 2]

Alta Cardinalidad (>50 categorías):

- ✓ **Frequency Encoding:** Reemplaza por frecuencia/porcentaje
- ✓ **Target Encoding:** Reemplaza por media del target
 - ⚠ Riesgo de data leakage, usar con validación cruzada
- ✓ **Binary Encoding:** Representación binaria

- ✓ **Hashing:** Para cardinalidad muy alta
- ✓ **Embedding Layers:** Para deep learning

5.2 Consideraciones Especiales:

- ✓ **Categorías no vistas en test:** Estrategia de manejo
- ✓ **Orden de aplicación:** Antes o después del split
- ✓ **Consistencia:** Mismo encoding en train/validation/test

Tarea	Snippet
One-Hot	<code>pd.get_dummies(df, columns=['cat'], drop_first=False)</code>
Dummy encoding	<code>pd.get_dummies(df, columns=['cat'], drop_first=True)</code>
Label Encoding	<code>from sklearn.preprocessing import LabelEncoder</code> <code>le=LabelEncoder(); df['cat_le']=le.fit_transform(df['cat'])</code>
Ordinal Encoding	<code>from sklearn.preprocessing import OrdinalEncoder</code> <code>oe=OrdinalEncoder(categories=[['Bajo', 'Medio', 'Alto']]);</code> <code>df[['cat_ord']]=oe.fit_transform(df[['cat']])</code>
Frequency Encoding	<code>freq=df['cat'].value_counts(); df['cat_freq']=df['cat'].map(freq)</code>
Target Encoding	<code>target_mean=df.groupby('cat')['target'].mean();</code> <code>df['cat_te']=df['cat'].map(target_mean)</code>
Binary Encoding	<code>category_encoders.BinaryEncoder(cols=['cat']).fit_transform(df)</code>
Hashing Encoding	<code>category_encoders.HashingEncoder(cols=['cat'],</code> <code>n_components=8).fit_transform(df)</code>

FASE 6: TRATAMIENTO DE OUTLIERS

6.1 Detección:

- ✓ **IQR (Rango Intercuartílico):** $Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$
- ✓ **Z-Score:** $|z| > 3$ (distribución normal)
- ✓ **Isolation Forest:** Para outliers multivariados
- ✓ **DBSCAN:** Clustering para detectar anomalías
- ✓ **Dominio experto:** Conocimiento del negocio

6.2 Tratamiento:

- ✓ **Mantener:** Si son valores válidos e informativos
- ✓ **Eliminar:** Si son errores claros ($< 1-5\%$ datos)

- ✓ **Winsorizar:** Reemplazar por percentiles (ej: p5, p95)
- ✓ **Transformar:** Log, sqrt para reducir impacto
- ✓ **Imputar:** Como valor faltante y aplicar imputación
- ✓ **Agrupar:** Crear categoría "extremo"

Tarea	Snippet
Isolation Forest	<pre>iso=IsolationForest(contamination=0.01); outliers=iso.fit_predict(df[['x']])</pre>
DBSCAN	<pre>from sklearn.cluster import DBSCAN db=DBSCAN(eps=0.5,min_samples=5); labels=db.fit_predict(df[['x','y']])</pre>
Winsorizar	<pre>df['x']=mstats.winsorize(df['x'], limits=[0.01,0.01])</pre>
Clipping	<pre>df['x']=df['x'].clip(lower=df['x'].quantile(0.01), upper=df['x'].quantile(0.99))</pre>

FASE 7: MANEJO DE DESBALANCEO (si aplica)

7.1 Para Clasificación Desbalanceada:

Análisis:

- ✓ **Ratio de desbalanceo:** Clase mayoritaria / minoritaria
- ✓ **Impacto esperado** en el modelo

Técnicas de Balanceo:

- ✓ **Undersampling:** Reducir clase mayoritaria
 - Random undersampling
 - Tomek links
 - NearMiss
- ✓ **Oversampling:** Aumentar clase minoritaria
 - Random oversampling
 - **SMOTE** (Synthetic Minority Over-sampling)
 - ADASYN
- ✓ **Híbridos:** SMOTE + Tomek links

Alternativas:

- ✓ **Pesos de clase:** class_weight='balanced'
- ✓ **Ajuste de threshold:** Cambiar umbral de decisión
- ✓ **Métricas apropiadas:** F1, Precision-Recall, ROC-AUC

Tarea	Snippet
Ratio	<code>ratio=df['target'].value_counts().max()/df['target'].value_counts().min()</code>
Random undersampling	<code>from imblearn.under_sampling import RandomUnderSampler rus=RandomUnderSampler(); X_r,y_r=rus.fit_resample(X,y)</code>
SMOTE	<code>from imblearn.over_sampling import SMOTE sm=SMOTE(); X_sm,y_sm=sm.fit_resample(X,y)</code>
ADASYN	<code>from imblearn.over_sampling import ADASYN ada=ADASYN(); X_ad,y_ad=ada.fit_resample(X,y)</code>
Pesos de clase	<code>model=RandomForestClassifier(class_weight='balanced')</code>

FASE 8: ESCALADO Y NORMALIZACIÓN

8.1 Cuándo Escalar:

- ✓ **NECESARIO:** Regresión Lineal/Logística, SVM, KNN, Neural Networks, PCA
- ✓ **NO NECESARIO:** Tree-based models (Random Forest, XGBoost, Decision Trees)

8.2 Métodos de Escalado:

StandardScaler (Estandarización):

- ✓ Fórmula: $(x - \text{media}) / \text{std}$
- ✓ Resultado: Media = 0, Std = 1
- ✓ Usar cuando: Distribución aprox. normal, sin outliers extremos
- ✓ **MÁS COMÚN**

MinMaxScaler (Normalización):

- ✓ Fórmula: $(x - \text{min}) / (\text{max} - \text{min})$
- ✓ Resultado: Rango [0, 1] o [-1, 1]
- ✓ Usar cuando: Necesitas rango específico, datos con límites naturales
- ✓ Sensible a outliers

RobustScaler:

- ✓ Fórmula: $(x - \text{mediana}) / \text{IQR}$
- ✓ Resultado: Centrado en mediana
- ✓ Usar cuando: **Muchos outliers**
- ✓ **Robusto a valores extremos**

MaxAbsScaler:

- ✓ Fórmula: $x / |\max(x)|$
- ✓ Resultado: Rango [-1, 1]
- ✓ Usar cuando: Datos dispersos (sparse data)

Normalizer:

- ✓ Escala cada muestra (fila) a norma unitaria
- ✓ Usar cuando: Importa dirección, no magnitud

8.3 Aplicación Correcta:

- ✓ **FIT solo en train:** `scaler.fit(X_train)`
- ✓ **TRANSFORM en train y test:** `X_train_scaled = scaler.transform(X_train)`
- ✓ **NUNCA fit en test:** Evita data leakage
- ✓ **Guardar el scaler:** Para uso en producción

Tarea	Snippet
StandardScaler	<pre>from sklearn.preprocessing import StandardScaler sc=StandardScaler(); X_train=sc.fit_transform(X_train); X_test=sc.transform(X_test)</pre>
MinMaxScaler	<pre>from sklearn.preprocessing import MinMaxScaler mm=MinMaxScaler(); X_train=mm.fit_transform(X_train)</pre>
RobustScaler	<pre>from sklearn.preprocessing import RobustScaler rs=RobustScaler(); X_train=rs.fit_transform(X_train)</pre>
Guardar scaler	<pre>import joblib; joblib.dump(sc,'scaler.gz') sc=joblib.load('scaler.gz')</pre>
Normalizer (Por muestra)	<pre>Normalizer().transform()</pre>

🔍 FASE 9: SELECCIÓN DE CARACTERÍSTICAS

9.1 Métodos de Selección:

Filter Methods (Independientes del modelo):

- ✓ **Correlación con target:** Pearson, Spearman
- ✓ **Test estadísticos:** Chi-cuadrado, ANOVA, Mutual Information
- ✓ **Varianza:** Eliminar features con varianza casi cero
- ✓ **VIF:** Eliminar features muy correlacionadas ($VIF > 10$)

Wrapper Methods (Dependientes del modelo):

- ✓ **Recursive Feature Elimination (RFE):** Elimina iterativamente
- ✓ **Forward Selection:** Añade features incrementalmente
- ✓ **Backward Elimination:** Elimina features incrementalmente

Embedded Methods (Durante entrenamiento):

- ✓ **Feature Importance:** Tree-based models
- ✓ **Regularización:** Lasso (L1), Ridge (L2), Elastic Net
- ✓ **Coeficientes:** Regresión lineal/logística

9.2 Reducción de Dimensionalidad:

- ✓ **PCA:** Componentes principales (no supervisado)
- ✓ **LDA:** Linear Discriminant Analysis (supervisado)
- ✓ **t-SNE:** Visualización (no para modelado)
- ✓ **UMAP:** Alternativa moderna a t-SNE

9.3 Criterios de Decisión:

- ✓ **Importancia predictiva vs interpretabilidad**
- ✓ **Costo computacional vs precisión**
- ✓ **Número objetivo de features** (regla: $n_{\text{features}} < \sqrt{n_{\text{samples}}}$)

Tarea	Snippet
Correlación con target	<code>corr=df.corr(numeric_only=True)['target'].abs().sort_values(ascending=False)</code>
Chi-cuadrado	<code>from sklearn.feature_selection import chi2, SelectKBest sel=SelectKBest(chi2,k=10); X_new=sel.fit_transform(X,y)</code>
Mutual Information	<code>from sklearn.feature_selection import mutual_info_classif mi=mutilual_info_classif(X,y); idx=np.argsort(mi)[-10:]</code>
VIF	<code>from statsmodels.stats.outliers_influence import variance_inflation_factor vif=pd.Series([variance_inflation_factor(X.values,i) for i in range(X.shape[1])], index=X.columns)</code>
RFE	<code>from sklearn.feature_selection import RFE rfe=RFE(RandomForestClassifier(),n_features_to_select=10); X_rfe=rfe.fit_transform(X,y)</code>
Lasso	<code>from sklearn.linear_model import LassoCV lasso=LassoCV(cv=5).fit(X,y); coef=pd.Series(lasso.coef_,index=X.columns);</code>

	<code>selected=coef[coef!=0].index</code>
PCA	<code>from sklearn.decomposition import PCA</code> <code>pca=PCA(n_components=0.95); X_pca=pca.fit_transform(X)</code>

🔗 □ FASE 10: DIVISIÓN DE DATOS

10.1 Estrategias de División:

Train/Test Split:

- ✓ **Ratio típico:** 80/20 o 70/30
- ✓ **Estratificación:** Para clasificación (stratify=y)
- ✓ **Random state:** Para reproducibilidad
- ✓ **Shuffle:** Mezclar antes de dividir (si no hay dependencia temporal)

Train/Validation/Test:

- ✓ **Ratio típico:** 60/20/20 o 70/15/15
- ✓ **Uso:**
 - Train: Entrenar modelo
 - Validation: Ajustar hiperparámetros
 - Test: Evaluación final (nunca visto por el modelo)

Consideraciones Temporales:

- ✓ **Series temporales:** Split cronológico (NO shuffle)
- ✓ **Time series CV:** Forward chaining
- ✓ **Sliding window:** Para datos secuenciales

10.2 Validación Cruzada:

- ✓ **K-Fold CV:** k=5 o k=10 típicamente
- ✓ **Stratified K-Fold:** Para clasificación
- ✓ **Time Series Split:** Para series temporales
- ✓ **Leave-One-Out:** Para datasets muy pequeños
- ✓ **Group K-Fold:** Cuando hay grupos que deben permanecer juntos

Tarea	Snippet
Train/test estratificado	<code>from sklearn.model_selection import train_test_split</code> <code>X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,stratify=y,random_state=42)</code>

Train/val/ test	<pre>X_train,X_temp,y_train,y_temp=train_test_split(X,y,test_size=0.3,stratify=y,random_state=42) X_val,X_test,y_val,y_test=train_test_split(X_temp,y_temp,test_size=0.5, stratify=y_temp,random_state=42)</pre>
TimeSeriesSplit	<pre>from sklearn.model_selection import TimeSeriesSplit tscv=TimeSeriesSplit(n_splits=5)</pre>
StratifiedKFold	<pre>from sklearn.model_selection import StratifiedKFold cv=StratifiedKFold(n_splits=5,shuffle=True,random_state=42)</pre>

FASE 11: VALIDACIÓN DEL PREPROCESAMIENTO

11.1 Checks de Calidad:

- ✓ **Sin data leakage:** Verificar que info de test no filtra a train
- ✓ **Consistencia:** Mismo preprocesamiento en train/test
- ✓ **Shapes:** Verificar dimensiones correctas
- ✓ **No valores faltantes inesperados**
- ✓ **Rangos correctos** después de transformaciones

11.2 Evaluación con Modelo Baseline:

- ✓ **Modelo simple:** Random Forest, Logistic Regression
- ✓ **Métricas apropiadas:** Según el problema
 - Clasificación: Accuracy, Precision, Recall, F1, ROC-AUC
 - Regresión: MAE, MSE, RMSE, R^2 , MAPE
- ✓ **Comparación:** Con/sin preprocesamiento
- ✓ **Feature importance:** Validar features creadas

11.3 Análisis de Resultados:

- ✓ **Mejora respecto a baseline**
- ✓ **Features más importantes**
- ✓ **Overfitting/Underfitting**
- ✓ **Errores sistemáticos**

Tarea	Snippet
Reproducibilidad completa	<pre>import joblib, hashlib joblib.dump({'X_train':X_train,'y_train':y_train},'train_set.pkl') hashlib.md5(open('train_set.pkl','rb').read()).hexdigest()</pre>

Modelo baseline	<pre> from sklearn.metrics import classification_report rf=RandomForestClassifier(n_estimators=200,random_state=42).fit(X_train,y_train) print(classification_report(y_test,rf.predict(X_test))) </pre>
Comparación antes/después	<pre> acc_sin_prep = cross_val_score(rf_sin,X_raw,y,cv=5).mean() acc_con_prep = cross_val_score(rf_con,X_prep,y,cv=5).mean() </pre>

FASE 12: PIPELINE Y PRODUCCIÓN

12.1 Creación de Pipeline:

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

# Pipeline completo y reproducible
pipeline = Pipeline([
    ('preprocessor', ColumnTransformer([
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(), categorical_features)
    ])),
    ('model', RandomForestClassifier())
])

```

12.2 Serialización:

- ✓ **Guardar objetos:** pickle, joblib
- ✓ **Versioning:** Registrar versión del preprocesamiento
- ✓ **Documentación:** Decisiones tomadas y por qué

12.3 Monitoreo en Producción:

- ✓ **Data drift:** Cambios en distribución de entrada
- ✓ **Concept drift:** Cambios en relación X-y
- ✓ **Valores fuera de rango:** Nuevas categorías, outliers extremos
- ✓ **Degradación del modelo**

Tarea	Snippet
Pipeline completo	<pre> from sklearn.compose import ColumnTransformer from sklearn.pipeline import Pipeline numeric_pipe=Pipeline([('imputer',SimpleImputer(strategy='median')),('scaler',StandardScaler())]) categorical_pipe=Pipeline([('imputer',SimpleImputer(strategy='most_frequent </pre>

	<pre> '')),('onehot',OneHotEncoder(handle_unknown='ignore'))])) pre=ColumnTransformer([('num',numeric_pipe,num_cols),('cat',categorical_pipe,cat_cols)]) clf=Pipeline([('prep',pre),('model',RandomForestClassifier(class_weight='balanced',random_state=42))]) </pre>
Guardar pipeline	<pre> joblib.dump(clf,'full_pipeline.pkl') </pre>
Cargar y predecir	<pre> model=joblib.load('full_pipeline.pkl') pred=model.predict(new_data) </pre>
Monitoreo de drift	<pre> from evidently import ColumnMapping from evidently.report import Report report=Report(metrics=[DataDriftPreset()]); report.run(reference_data=ref,current_data=curr) report.save_html('drift_report.html') </pre>

CHECKLIST FINAL

☒ Antes de Modelar:

- ☐ EDA completo realizado y documentado
- ☐ Valores faltantes tratados con estrategia justificada
- ☐ Outliers analizados y tratados apropiadamente
- ☐ Features creadas y validadas
- ☐ Encoding apropiado para cada tipo de variable
- ☐ Escalado aplicado correctamente (fit en train, transform en test)
- ☐ División de datos con estratificación (si aplica)
- ☐ Sin data leakage confirmado
- ☐ Pipeline reproducible creado
- ☐ Baseline model evaluado

☒ Documentación:

- ☐ Decisiones de preprocesamiento documentadas
- ☐ Código limpio y comentado
- ☐ Resultados de EDA guardados

- [] Métricas de validación registradas
 - [] Artefactos guardados (scalers, encoders, etc.)
-

MEJORES PRÁCTICAS

Generales:

1. **SIEMPRE dividir primero**, preprocesar después
2. **NUNCA hacer fit en test set**
3. **Documentar todas las decisiones** y su justificación
4. **Usar pipelines** para reproducibilidad
5. **Validar con CV** antes de test final
6. **Comparar múltiples estrategias** de preprocesamiento
7. **Mantener simplicidad** cuando sea posible
8. **Entender el dominio** antes de tomar decisiones

Evitar:

- ✗ Data leakage en cualquiera de sus formas
 - ✗ Eliminar outliers sin análisis
 - ✗ Imputar sin entender el mecanismo de faltantes
 - ✗ Escalar cuando no es necesario
 - ✗ One-hot encoding para alta cardinalidad sin análisis
 - ✗ Target encoding sin validación cruzada
 - ✗ Eliminar features sin validar importancia
-

RECURSOS RECOMENDADOS

- **Scikit-learn Documentation:** Referencia oficial de preprocesamiento
 - **Feature Engineering Book** (Alice Zheng & Amanda Casari)
 - **Kaggle Notebooks:** Best practices de la comunidad
 - **Papers:** SMOTE, Isolation Forest, Feature Selection methods
 - **Blogs:** Towards Data Science, Machine Learning Mastery
-